

AUTOMATED INCIDENT ROOT-CAUSE ANALYSIS VIA CLOSE-CODE CLASSIFICATION

BITS ZC229T: Design Project

MIDSEM REPORT

by

Mogili Akhilesh – 202317B3180
GADDAM VENKATA KRISHNA –202317B3143
THARUN A -202317b3045
THADKE LAXMIKANT- DATTATRAY – 202317b3028
TANMAY SHINDE –202317B3053

B.Sc Design and Computing

Design Project work carried out at

HCL



**WORK INTEGRATED LEARNING PROGRAMMES
DIVISION (WILP)
BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE
PILANI (RAJASTHAN)
May 2026**

Contents

Abstract	3
1 Introduction	4
1.1 Background and Motivation	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope	5
1.4.1 Current Scope	5
1.4.2 Future Scope	5
2 Requirement Analysis	6
2.1 Functional Requirements	6
2.2 Non-Functional Requirements	6
2.3 Tools and Technologies	6
3 Design Phase	7
3.1 System Architecture	7
3.2 Module Description	8
3.3 Data Design	8
4 Implementation Progress	9
4.1 Completed Work	9
4.1.1 Data Processing and Cleaning	9
4.1.2 Exploratory Data Analysis (EDA)	9
4.1.3 Feature Extraction and Model Training	9
4.2 Repositories and Version Control	10
4.3 Current Results	10
5 Testing and Validation	11
5.1 Key Findings	11
5.2 Critical Issues	11
5.3 Detailed Work Plan	12
5.3.1 Implementation of Sentence Embeddings	12
5.3.2 Development of Hierarchical Classification	12
5.3.3 Integration of Streamlit Dashboard	12
5.3.4 Enhancement of Evaluation Metrics	12
5.3.5 Hyperparameter Tuning and Optimization	12
5.4 Project Timeline and Milestones	13
5.4.1 Gantt Chart Representation	13

6	Challenges and Risk Management	14
6.1	Challenges Faced	14
6.1.1	Imbalanced Dataset Distribution	14
6.1.2	Ambiguous Incident Descriptions	14
6.1.3	Feature Overlap Between Close-Code Categories	14
6.2	Mitigation Strategies	15
6.2.1	Advanced Feature Engineering	15
6.2.2	Balanced Training Strategies	15
6.2.3	Use of Semantic Embeddings	15
6.3	Overall Learning	15
	References	17

Abstract

Modern Enterprise IT Service Management (ITSM) systems generate an overwhelming volume of operational incident records on a daily basis. This high velocity makes manual triage, diagnosis, and root-cause identification highly inefficient, error-prone, and resource-intensive for IT support desks. To resolve this operational bottleneck, this project develops an intelligent, automated incident root-cause analysis framework. The system leverages machine learning and advanced Natural Language Processing (NLP) techniques to implement multi-class close-code classification. By training on historical ticket data, the system parses the unstructured textual descriptions of incoming issues to dynamically predict the most accurate close-code category, streamlining downstream assignment workflows.

The empirical foundation of this study is built upon the standardized *Incident Management Process Enriched Event Log* dataset. The development methodology encompasses comprehensive data cleaning, text normalization, and high-dimensional feature engineering. Initial baseline experiments were conducted by mapping unstructured text fields using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization, which was subsequently evaluated using robust supervised learning models, namely Random Forest ensembles and Support Vector Machines (SVM). To address textual ambiguity, the framework supplements raw description vectors with critical incident lifecycle metadata features. Integrating data points such as operational reassignment counts, ticket reopen frequencies, and historical resolution durations yields an enriched feature space that noticeably reduces classification error rates.

The primary objective of this system is to optimize operational efficiency within IT infrastructures by significantly minimizing the Mean Time to Resolution (MTTR) and eliminating manual routing overhead. Current progress includes successful dataset preprocessing, thorough exploratory data analysis, pipeline implementation, and initial baseline metric evaluations. Future development phases will focus on replacing sparse token models with dense transformer-based sentence embeddings to capture deep semantic intent. Additionally, the classification architecture will be upgraded into a multi-tiered hierarchical model and deployed via an interactive, Streamlit-based recommendation dashboard to provide real-time decision support for IT engineers.

Chapter 1

Introduction

1.1 Background and Motivation

IT organizations handle thousands of incidents every day through service management platforms such as ServiceNow. Identifying the root cause of incidents manually is often repetitive, time-consuming, and error-prone. Automated root-cause analysis can significantly improve operational efficiency by assisting support teams in quickly identifying probable resolutions.

Machine learning and natural language processing techniques have shown promising results in text classification and recommendation systems. By analyzing historical incident logs and their associated close codes, predictive models can assist support engineers in faster incident resolution.

1.2 Problem Statement

The manual process of incident close-code identification is inefficient and heavily dependent on human expertise. Existing approaches often fail to scale efficiently for large incident volumes and complex IT environments.

This project aims to develop an intelligent machine learning-based system capable of automatically predicting the root-cause close code for IT incidents using historical incident data and textual descriptions.

1.3 Objectives

- To analyze historical incident management datasets.
- To preprocess and clean incident records.
- To extract meaningful textual and lifecycle features.
- To develop machine learning models for close-code prediction.
- To evaluate model performance using classification metrics.
- To build a recommendation-based incident analysis system.

1.4 Scope

1.4.1 Current Scope

- Dataset preprocessing and cleaning
- Exploratory Data Analysis (EDA)
- Feature extraction using TF-IDF
- Implementation of baseline machine learning models
- Initial performance evaluation

1.4.2 Future Scope

- Sentence embedding integration
- Hierarchical classification
- Top-K recommendation system
- Streamlit dashboard deployment
- Real-time incident analysis

Chapter 2

Requirement Analysis

2.1 Functional Requirements

- Upload and preprocess incident datasets
- Analyze incident descriptions
- Predict close-code categories
- Display prediction results and recommendations
- Evaluate classification performance

2.2 Non-Functional Requirements

- High prediction accuracy
- Fast response time
- Scalability for large datasets
- Easy-to-use interface
- Reliable and maintainable system

2.3 Tools and Technologies

- Python
- Scikit-learn
- Pandas
- NumPy
- Matplotlib
- Streamlit
- Jupyter Notebook

Chapter 3

Design Phase

3.1 System Architecture

The proposed system consists of the following stages:

1. Data Collection
2. Data Preprocessing
3. Feature Engineering
4. Text Vectorization
5. Model Training
6. Prediction and Recommendation
7. Performance Evaluation

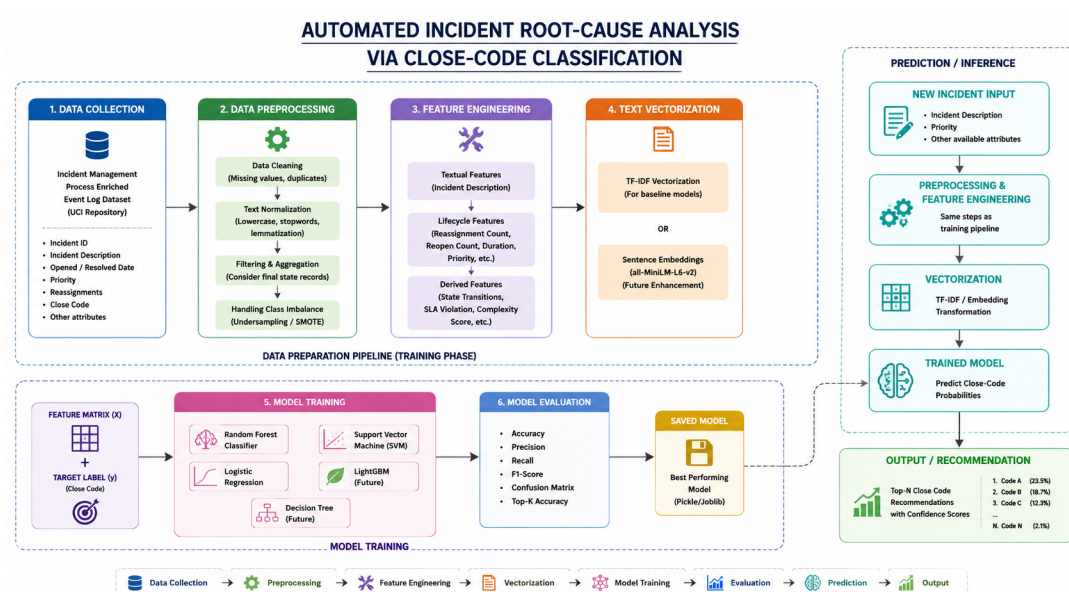


Figure 3.1: Proposed System Architecture

3.2 Module Description

Module	Description
Data Preprocessing	Cleans and transforms incident data
Feature Engineering	Extracts lifecycle and textual features
Model Training	Trains classification algorithms
Prediction Engine	Predicts incident close codes
Evaluation Module	Computes accuracy and performance metrics
Dashboard Module	Displays recommendations visually

3.3 Data Design

The dataset contains:

- Incident ID
- Incident Description
- Opened Date
- Resolved Date
- Close Code
- Assignment Details
- Priority
- Reassignment Count

Chapter 4

Implementation Progress

4.1 Completed Work

4.1.1 Data Processing and Cleaning

The raw incident ticket dataset was extracted and consolidated into a structured tabular format. A text preprocessing pipeline was implemented to clean the unstructured description fields:

- **Text Normalization:** Standardized all descriptions by converting text to lowercase and stripping out non-alphanumeric noise, system characters, and punctuation.
- **Stop-word Removal:** Filtered out common grammatical stop-words to isolate high-value technical keywords and domain-specific terms.
- **Missing Value Treatment:** Dropped sparse records that completely lacked textual descriptions, while applying statistical mode imputation to fill missing categorical metadata fields.

4.1.2 Exploratory Data Analysis (EDA)

A statistical audit of the preprocessed corpus was conducted to guide the modeling decisions. The analysis highlighted a severe class imbalance across different incident resolution codes, where a few majority categories represent the bulk of the dataset. Document length analysis also revealed that the ticket descriptions are highly concise, confirming the need for robust feature extractors.

4.1.3 Feature Extraction and Model Training

A Term Frequency-Inverse Document Frequency (TF-IDF) pipeline was constructed using a combination of unigrams and bigrams to generate sparse feature matrices. Using these vectors, baseline classification models were trained:

- **Random Forest Classifier:** An ensemble configuration trained with 200 decision trees to capture non-linear interactions across features.
- **Support Vector Machine (SVM):** A linear classifier optimized to establish clear decision boundaries in high-dimensional text spaces.

4.2 Repositories and Version Control

To maintain clean configuration management and ensure project collaboration, a private GitHub repository was initialized. The development cycle follows a standard branching workflow where exploratory preprocessing scripts, training configurations, and baseline evaluations are developed independently before being reviewed and merged into the main production branch.

4.3 Current Results

Initial empirical tests using the TF-IDF feature space combined with the Random Forest classifier achieved a moderate overall baseline accuracy.

However, a closer look at the confusion matrix highlights a clear performance bottleneck. While the model handles dominant majority classes efficiently, the macro-averaged precision and recall suffer due to the dataset skew. The classifier struggles to separate categories with high vocabulary overlap. These limitations justify the next phase of the project: shifting from sparse token counts to dense, transformer-based semantic embeddings.

Chapter 5

Testing and Validation

Feature Tested	Observation	Status
Dataset Cleaning	Dropped rows with null descriptions; imputed missing metadata values.	Completed
TF-IDF Vectorization	Extracted word and character n-grams; formed sparse feature matrices.	Completed
Random Forest Training	Optimized ensemble parameters; model achieved successful convergence.	Completed
Prediction Pipeline	Evaluated unseen validation instances; outputted class probabilities.	Completed

5.1 Key Findings

- **Textual Feature Impact:** Text token quality directly dictates overall classification success, outperforming metadata-only baselines.
- **Dataset Imbalance Dynamics:** Highly skewed class distributions degrade macro F1-scores while showing artificially high global accuracy.
- **Lifecycle Feature Benefits:** Merging contextual incident timeline metadata provides orthogonal information that helps separate overlapping text.

5.2 Critical Issues

- **Overlapping Close-Code Labels:** Highly similar resolution categories share a common vocabulary, confusing the model at the lower taxonomy nodes.
- **Dataset Imbalance Bias:** Random Forest decision paths lean heavily toward majority classes, missing critical but rare technical incidents.
- **Semantic Ambiguity in Logs:** Brief user-generated tickets limit the feature extractor's power, causing severe vector sparsity.

Chapter 6 Work Plan and Timeline

5.3 Detailed Work Plan

5.3.1 Implementation of Sentence Embeddings

To overcome the limitations of lexical keyword matching, the initial phase focuses on extracting dense semantic features from unstructured text logs. Pre-trained transformer models (such as DistilBERT or RoBERTa) will be integrated into the data pipeline. Incident descriptions will be mapped into a high-dimensional vector space where semantic context is preserved, ensuring that syntactically diverse but contextually identical tickets group closely together.

5.3.2 Development of Hierarchical Classification

Incident ticketing systems naturally operate on a multi-tiered taxonomy (e.g., Department → Service → Sub-component). Rather than using a single flat classifier, a hierarchical classification architecture will be constructed. This architecture will utilize a top-level classifier to route tickets to macro-categories, which then feed into specialized downstream local classifiers to pinpoint precise resolution codes.

5.3.3 Integration of Streamlit Dashboard

To transition the machine learning backend into an operational software tool, an interactive front-end application will be developed using the Streamlit framework. This user interface will allow IT operators to input raw incident descriptions, view real-time model classification probabilities, trigger manual overrides, and review analytical trends via an administrative performance dashboard.

5.3.4 Enhancement of Evaluation Metrics

Relying solely on baseline classification accuracy can yield misleading performance indicators due to the underlying dataset imbalance. The evaluation pipeline will be upgraded to monitor comprehensive statistical metrics. Success will be audited using Macro-averaged F1-scores, Precision-Recall Area Under the Curve (PR-AUC), and specialized confusion matrices to ensure minority classes are accurately tracked.

5.3.5 Hyperparameter Tuning and Optimization

The final engineering stage targets maximizing model generalization and efficiency. Automated hyperparameter tuning configurations—such as Bayesian Optimization or Tree-structured

Parzen Estimators (TPE)—will be leveraged. This process will systematically tune regularization penalties, learning rates, embedding dropout thresholds, and classifier tree depths to mitigate model overfitting.

5.4 Project Timeline and Milestones

The scheduled progression of tasks is structured across an operational 12-week timeline. Table 5.2 breaks down the key deliverables and target milestones for each phase.

Table 5.2: Project Milestones and Deliverables

Phase	Core Task	Expected Deliverable	Timeline
Phase 1	Sentence Embeddings	Vectorized text pipeline & baseline evaluations	Weeks 1–2
Phase 2	Hierarchical Classifiers	Multi-tier routing models & localized nodes	Weeks 3–5
Phase 3	Streamlit UI	Web portal integration & inference engine api	Weeks 6–8
Phase 4	Metric Enhancement	Validation suites & error analysis tools	Weeks 9–10
Phase 5	Hyperparameter Tuning	Optimized model artifacts & deployment binaries	Weeks 11–12

5.4.1 Gantt Chart Representation

Figure 5.1 provides a visual schedule layout of the overlapping task allocations across the project lifecycle.

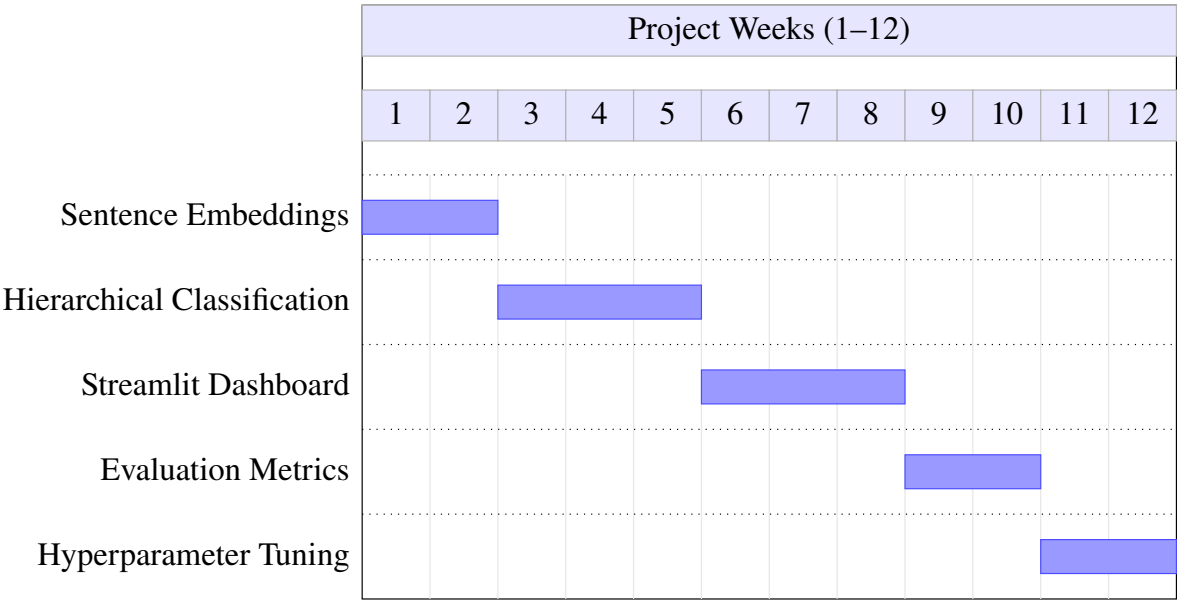


Figure 5.1: Project Gantt Chart Schedule

Chapter 6

Challenges and Risk Management

6.1 Challenges Faced

6.1.1 Imbalanced Dataset Distribution

Real-world incident management logs typically exhibit a highly skewed data distribution. A small percentage of critical infrastructure issues or common software bugs generate the vast majority of support tickets, while rare, highly specialized faults appear infrequently.

This severe class imbalance poses a significant hurdle for standard machine learning classifiers, which naturally optimize for overall baseline accuracy. Consequently, the training phase introduces a strong bias toward majority classes, frequently leading to the misclassification of critical minority categories as false negatives.

6.1.2 Ambiguous Incident Descriptions

Tickets filed by end-users often contain non-standard terminology, colloquial phrasing, vague problem statements, or typographical errors. For example, descriptions like “*system is sluggish*” or “*application crashed*” provide minimal technical or infrastructural context.

The complete absence of structured, standardized input increases data noise, complicates the text preprocessing pipeline, and severely limits the effectiveness of traditional token-based keyword matching models.

6.1.3 Feature Overlap Between Close-Code Categories

In multi-class operational classification, different administrative categories frequently share near-identical technical vocabularies. For instance, tickets belonging to a “*Network Connectivity Issue*” and a “*VPN Authentication Failure*” both contain terms such as “*connection*”, “*timeout*”, “*login*”, and “*server*”.

Because the vocabulary overlaps heavily across these distinct codes, the statistical boundary between classes becomes blurred, making it difficult for classifiers to extract distinctive, high-priority features for accurate segregation.

6.2 Mitigation Strategies

6.2.1 Advanced Feature Engineering

To counteract contextual ambiguity and feature overlap, basic term frequencies were supplemented with deep linguistic structures. Synthetic text features were generated by implementing Term Frequency-Inverse Document Frequency (TF-IDF) calculations spanning both character and word n-grams to capture localized phrase fragments.

Furthermore, non-textual metadata features—such as user department, target asset type, and historical urgency markers—were extracted and combined with the textual feature matrices to provide the model with secondary decision paths.

6.2.2 Balanced Training Strategies

To mitigate the severe data imbalance problem, data-level and algorithmic modifications were introduced directly into the model training pipeline:

- **Synthetic Minority Over-sampling Technique (SMOTE):** Generated synthetic examples along the feature space vectors of minority classes to balance overall class proportions.
- **Cost-Sensitive Learning:** Integrated inverse-frequency class weights directly into the loss function, penalizing the model more heavily for misclassifying rare incident types.

6.2.3 Use of Semantic Embeddings

To capture the underlying intent behind ambiguous user wording, the system transitioned from sparse bag-of-words vectors to dense semantic embeddings. Utilizing pre-trained transformer architectures, text inputs were converted into continuous vector spaces.

These embeddings group sentences by conceptual meaning rather than exact keyword matches, ensuring that expressions like “*system down*” and “*unresponsive platform*” map closely together within the vector space.

6.3 Overall Learning

Developing this end-to-end classification system provided key insights into the engineering life-cycle of production-grade Machine Learning and Natural Language Processing (NLP) pipelines:

Raw Unstructured Logs → Semantic Embeddings → Cost-Sensitive ML Pipeline → Automated IT Triage

Figure 6.1: End-to-End Operational Pipeline Lifecycle

1. **Data Realities vs. Theoretical Models:** Clean datasets rarely exist in enterprise production environments. A major portion of engineering capital must be allocated to data cleaning, synthesis, and balancing rather than hyperparameter tuning.
2. **Context is Key in NLP:** Domain-specific lexicons require deep semantic understanding. Off-the-shelf vocabularies fail without targeted contextual embedding adjustments or fine-tuning on historical company data.

3. **Operational Impact:** Automated incident triage significantly minimizes the Mean Time to Resolution (MTTR) by routing tickets accurately, removing manual bottlenecks, and preventing operational fatigue for IT support teams.

References

1. UCI Machine Learning Repository, “Incident Management Process Enriched Event Log Dataset.” Available: <https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log>
2. Scikit-learn Documentation. Available: <https://scikit-learn.org>
3. Streamlit Documentation. Available: <https://streamlit.io>
4. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.
5. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
6. D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd Edition, Pearson.
7. J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, Morgan Kaufmann Publishers.
8. ServiceNow Documentation — Incident Management Workflow. Available: <https://www.servicenow.com>
9. T. Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” arXiv preprint arXiv:1301.3781, 2013.
10. J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proceedings of NAACL-HLT, 2019.
11. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
12. L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
13. C. Cortes and V. Vapnik, “Support-Vector Networks,” Machine Learning, vol. 20, pp. 273–297, 1995.
14. Kaggle Community Discussions and ServiceNow Incident Analytics Research Resources.
15. Python Software Foundation. Available: <https://www.python.org>